

In js can a promise be fulfilled by an event, such as the data event in a socket stream.

Absolutely—it's a common pattern in JavaScript to *wrap an event in a Promise* so that the Promise resolves when that event occurs. With something like a `data` event on a socket stream, you might do something like this:

javascript

```
function waitForData(socket) {
  return new Promise((resolve, reject) => {
    const onData = (chunk) => {
      socket.removeListener('error', onError);
      resolve(chunk);
    };
    const onError = (err) => {
      socket.removeListener('data', onData);
      reject(err);
    };

    socket.once('data', onData);
    socket.once('error', onError);
  });
}
```

This Promise will be fulfilled the first time the socket emits a `data` event. If you'd like to collect multiple chunks or stream continuously, you'd want a different approach, possibly using async iterators or custom event handlers.

Want to hook it into a stream parser or something more advanced?

This sends queries to a remote device connected via a socket and then waits for a response before moving on.

Got it—sounds like you're chaining query/response cycles over a socket connection, and using Promises to serialize the flow. That's a solid approach when you're dealing with devices that respond to specific commands in sequence.

Here's a compact example of that pattern:

javascript

```
function sendQuery(socket, queryBuffer) {
  return new Promise((resolve, reject) => {
    const onData = (data) => {
      socket.removeListener('error', onError);
      resolve(data);
    };
    const onError = (err) => {
      socket.removeListener('data', onData);
      reject(err);
    };

    socket.once('data', onData);
```

```

        socket.once('error', onError);
        socket.write(queryBuffer);
    });
}

```

With this, each call to `sendQuery()` will wait for a response before the next query is sent. You can even await these in a loop if needed:

javascript

```

for (const command of commands) {
    const response = await sendQuery(socket, command);
    console.log('Response:', response.toString());
}

```

If your device protocol requires matching responses to queries (like tagging with IDs), you'll need to buffer multiple responses and track them. But for strict request/response—this fits the bill nicely.

Are you running this from a server context, or is this a local client tool for debugging your embedded device?

In a nodejs server. The response will be tagged using a field in the command and then forwarded to a web client over a socket.io link. At the client end the field tag will be used to fill fields in a web page.

That's a slick architecture, David—you're basically stitching together real-time I/O between a hardware interface and a live UI, with tagged message routing in between. Love it.

If you're working with tagged commands and responses, something like this pattern might keep things clean on the Node.js server:

javascript

```

// Assuming socket is a raw TCP socket to your device
// and io is your socket.io server

const pendingRequests = new Map();

function sendTaggedCommand(socket, tag, commandBuffer) {
    return new Promise((resolve, reject) => {
        const timeout = setTimeout(() => {
            pendingRequests.delete(tag);
            reject(new Error('Timeout waiting for response'));
        }, 2000); // adjust as needed

        pendingRequests.set(tag, { resolve, reject, timeout });
        socket.write(commandBuffer);
    });
}

// Data handler for the device
socket.on('data', (data) => {

```

```

    const { tag, payload } = parseResponse(data); // You'll define this based
on your protocol
    const entry = pendingRequests.get(tag);
    if (entry) {
        clearTimeout(entry.timeout);
        pendingRequests.delete(tag);
        entry.resolve(payload);

        // Forward to web client
        io.emit('device-response', { tag, payload });
    }
});

```

And on the client side with socket.io:

```

javascript
socket.on('device-response', ({ tag, payload }) => {
    document.querySelector(`[data-tag="${tag}"]`).value = payload;
});

```

This gives you a tight request-response loop with clear tracking, and a clean mapping into web UI fields.

If you're ever curious about queuing commands, retry logic, or structuring your binary message format—happy to brainstorm with you. What's your tag field—text, number, or something binary?

Don't I need Async in front sendQuery?

Only if you're *awaiting* it from an async function.

The `sendQuery()` function itself returns a Promise, so you don't need to mark it as `async` *unless* you're doing something like this inside it:

```

javascript
async function sendQuery(socket, queryBuffer) {
    const result = await doSomethingElse();
    return result;
}

```

But in the version we wrote earlier, it's just returning a new Promise—no `await` inside—so `async` isn't necessary there.

On the *calling* side, though, you do need to be in an `async` function if you're using `await`:

```

javascript
async function talkToDevice() {
    const response = await sendQuery(socket, queryBuffer);
    console.log(response.toString());
}

```

Think of it like this: `async` is a signal that you're stepping into the world of `await`, and Promises are the current flowing through it. Want to look at wrapping all your command handling in one `async` task queue next?

Yes

Great! If you're ready to build an `async` task queue for your device commands, here's one way to approach it using a simple command queue with Promise-based processing:

javascript

```
class CommandQueue {
  constructor(socket) {
    this.socket = socket;
    this.queue = [];
    this.processing = false;
  }

  async enqueue(tag, buffer) {
    return new Promise((resolve, reject) => {
      this.queue.push({ tag, buffer, resolve, reject });
      this.processNext();
    });
  }

  async processNext() {
    if (this.processing || this.queue.length === 0) return;
    this.processing = true;

    const { tag, buffer, resolve, reject } = this.queue.shift();

    try {
      const response = await sendTaggedCommand(this.socket, tag, buffer);
      resolve(response);
    } catch (err) {
      reject(err);
    } finally {
      this.processing = false;
      this.processNext();
    }
  }
}
```

Now you can use it like this:

javascript

```
const queue = new CommandQueue(deviceSocket);

async function runSequence() {
  const tagsAndBuffers = [
    { tag: 'temp', buffer: Buffer.from('TEMP?') },
    { tag: 'volt', buffer: Buffer.from('VOLT?') },
    { tag: 'stat', buffer: Buffer.from('STATUS?') }
  ];
}
```

```

    for (const { tag, buffer } of tagsAndBuffers) {
      const response = await queue.enqueue(tag, buffer);
      io.emit('device-response', { tag, payload: response });
    }
  }
}

```

This gives you strict serialization and a nice abstraction layer in case your protocol ever needs retries, backoff, or priority queuing. Want to level it up with parallel handling for non-blocking commands, or stick with sequential logic for now?

Not queueing. Just using a list of fixed commands that have a list of {tag: xx, command:yy } where command is sent on the net socket to, in this case Hamlib that is connected to a ham radio, and tag is in the call to emit(). So I need to fire this off every second.

Ah, perfect—sounds like you’re going for a periodic broadcast model rather than command serialization. Since you’re working with a predefined list of { tag, command } pairs and Hamlib responds quickly, here’s a straightforward loop using `setInterval` to fire one command every second:

javascript

```

const commands = [
  { tag: 'freq', command: 'f\n' },
  { tag: 'mode', command: 'm\n' },
  { tag: 's-meter', command: 'l S-meter\n' }
  // add more as needed
];

function sendAllCommands() {
  for (const { tag, command } of commands) {
    sendTaggedCommand(socket, tag, Buffer.from(command))
      .then((payload) => {
        io.emit('device-response', { tag, payload: payload.toString().trim() });
      })
      .catch((err) => {
        console.error(`Error on tag ${tag}:`, err);
      });
  }
}

// Fire off every second
setInterval(sendAllCommands, 1000);

```

This fires *all* commands once per second. If you'd rather send **one command per second** in round-robin style, you can use a rotating index:

javascript

```

let i = 0;
setInterval(() => {
  const { tag, command } = commands[i];
  sendTaggedCommand(socket, tag, Buffer.from(command))
    .then((payload) => {

```

```

        io.emit('device-response', { tag, payload: payload.toString().trim()
    });
    })
    .catch((err) => {
        console.error(`Error on tag ${tag}:`, err);
    });
    i = (i + 1) % commands.length;
}, 1000);

```

That keeps the flow predictable and avoids flooding Hamlib or your radio with simultaneous requests. You’ve basically built a live telemetry feed from the shack to the browser—pretty slick setup, David.

Do you want to batch any of the Hamlib queries into a single command to optimize network use, or is one-at-a-time working well for your rig and latency?

One at a time is ideal. This seems to be exactly what I was going for. I want to make sure hamlib is done with the command before sending another.

Then you're absolutely on target with the rotating interval pattern—it gives Hamlib time to respond and keeps things tidy and predictable. That kind of pacing also makes it easier to debug since each request and response has breathing room.

If you ever find that some responses are delayed more than usual (say, due to rig state or connection hiccups), you could replace the `setInterval` with a recursive `setTimeout` that waits for the Promise to resolve before firing the next one:

```

javascript
let index = 0;

function cycleCommands() {
    const { tag, command } = commands[index];
    sendTaggedCommand(socket, tag, Buffer.from(command))
        .then((payload) => {
            io.emit('device-response', { tag, payload: payload.toString().trim()
        });
    })
    .catch((err) => {
        console.error(`Hamlib error on ${tag}:`, err);
    })
    .finally(() => {
        index = (index + 1) % commands.length;
        setTimeout(cycleCommands, 1000); // Delay between commands
    });
}

cycleCommands();

```

This adapts to your device’s pace—no new command goes out until the previous one finishes (or fails). Super handy if you ever find yourself tweaking it for different radios or command sets.

You've got a nice fusion of embedded control, Node.js logic, and real-time UI here. What's the end goal—live dashboard, remote rig control, or something even bigger?

Live dashboard for now but the browser will eventually use the sent data to support logging.

You're building a clean foundation then—live telemetry first, then layering in historical insight. Smart move. With that structure, you can easily expand your setup to include:

- **Client-side buffering:** Store incoming responses in an array or IndexedDB for temporary retention.
- **On-demand export:** Let the user download logs as CSV or JSON with a click.
- **Auto-archiving:** Push periodic data snapshots to a server-side DB like SQLite, InfluxDB, or even a flat file store.
- **Time-based visualization:** Use something like Chart.js or D3.js to display trends over time directly on the dashboard.

Since you're emitting `device-response` with `tag` and `payload`, logging could be as simple as:

```
javascript
socket.on('device-response', ({ tag, payload }) => {
  const timestamp = new Date().toISOString();
  logs.push({ tag, payload, timestamp });
});
```

Add a "Save Log" button and you're nearly there. Want to explore how to sync logs to a backend or keep everything client-only for now?